

Lecture 8

Inversion of Control

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Break Time

<https://bit.ly/4awBS1r>



Announcements

- Project 1 rubrics are posted on the website
- Project 2 due Friday
 - Feature demos in lab
(if your group members attend different labs, you only need to do it once)
 - Initial deliverables by 11:59pm

Inversion of Control

- Occurs when control (i.e. the flow of code execution) is established in the opposite order of dependency.
- Commonly used for user interfaces or reactive programming.
- Often expressed as “Don’t call us, we’ll call you”

Observer Pattern

- A design pattern often used for inversion of control.
- The **observed** entity maintains a list of **observers**.
 - Observers implement an interface specifying certain callbacks.
 - When the entity does something, it notifies observers via the callbacks.
 - The observed doesn't really care who the observers are.



Observer Pattern - Example

```
interface CarObserver {
  onSpeedUpdate(speed: number): void;
  onStopped(): void;
}

class Car {
  private observers: CarObserver[] = [];
  private speed: number = 0;

  // The Car (Subject) maintains a list of observers
  public addObserver(observer: CarObserver) {
    this.observers.push(observer);
  }

  public setSpeed(newSpeed: number) {
    const wasStopped = this.speed === 0;
    this.speed = newSpeed;

    // Notify all observers of the speed change
    this.observers.forEach(obs => obs.onSpeedUpdate(newSpeed));

    // If the car just came to a stop, notify observers
    if (newSpeed === 0 && !wasStopped) {
      this.observers.forEach(obs => obs.onStopped());
    }
  }
}
```

```
const car = new Car();

class Dashboard implements CarObserver {
  // ... HTML UI elements ...

  public constructor(container: HTMLElement) { /* not shown */ }

  public onSpeedUpdate(speed: number) {
    this.speedGauge.textContent = `${speed} mph`;
  }

  public onStopped() {
    this.statusIndicator.textContent = "STOPPED";
  }
}

const dashboard = new Dashboard(dashboardContainer);

const safetySystem: CarObserver = {
  onSpeedUpdate: (speed) => {
    if (speed > 80) console.log(`Safety: Warning! Too fast!`);
  },
  onStopped: () => {} // No action needed for safety system
};

car.addObserver(dashboard);
car.addObserver(safetySystem);
```

Publish/Subscribe (Pub/Sub) Pattern

- A design pattern often used for inversion of control.
- Like observer pattern, but through an intermediary “**broker**”.
 - The broker organizes events by “topic” (e.g. a string).
 - Subjects can publish events for a certain topic via the broker.
 - The broker broadcasts the events to any subscribers registered for the topic.
- Sometimes the broker exists as a singleton.

Pub/Sub Pattern - Example

```
type CarTopic = "speed" | "door_status" | "fuel_level" | ...;

interface Subscriber {
  onUpdate(data: string): void;
}

class Broker {
  private topics: { [key in CarTopic]: Subscriber[] } = {};

  // The "Broker" manages the communication
  subscribe(topic: CarTopic, cb: Subscriber) {
    this.topics[topic]!.push(cb);
  }

  publish(topic: CarTopic, data: string) {
    this.topics[topic]!.forEach(cb => cb.onUpdate(data));
  }
}

class Car {
  public constructor(private broker: Broker) { }

  public setSpeed(speed: number) {
    // ...
    this.broker.publish("speed", `${speed} mph`);
  }
}
```

```
// Dashboard subscribes to speed and fuel
class DashboardSubscriber implements Subscriber {
  // ... UI elements ...

  public constructor(container: HTMLElement) {
    // ... create UI elements, implementation not shown ...
  }

  onUpdate(data: string) {
    this.displayPanel.textContent = data;
  }
}

// Dashboard shows speed and fuel
const dashboardSubscriber = new
DashboardSubscriber(dashboardContainer2);
broker.subscribe("speed", dashboardSubscriber);
broker.subscribe("fuel_level", dashboardSubscriber);

// Security system only cares about doors
const securitySubscriber: Subscriber = {
  onUpdate: (data) => console.log(`Security system: ${data}`)
};
broker.subscribe("door_status", securitySubscriber);
```